Guia Passo a Passo — Estação ADS-B com Alertas no Telegram

Este guia ensina a montar do zero uma estação que **recebe sinais de aeronaves** (ADS-B / Modo S em 1090 MHz), **alimenta o ADS-B Exchange** e **envia alertas no Telegram** quando uma aeronave da sua lista é detectada.

> **Como ler este guia:** tudo que aparece entre `<` e `>` é um **marcador** que você deve substituir pelo seu próprio valor. Exemplo: `<SEU_TOKEN_AQUI>`. Nunca publique seus valores reais em fóruns, grupos ou repositórios.

---

1. O que você vai precisar

Item	Observação
Raspberry Pi 3 (1 GB) ou superior	O Pi 3 funciona, mas com pouca folga de memória
Cartão microSD de 16 GB ou mais	De boa qualidade (classe A1/A2)
Fonte 5V 2,5A original para Raspberry Pi	Fonte fraca causa travamentos e corrupção do cartão
RTL-SDR V3 (ou clone com chip RTL2832U)	Receptor de rádio por software
Filtro passa-faixa 1090 MHz	Reduz interferência de outras frequências
Antena para 1090 MHz	Quanto mais alta e desobstruída, melhor o alcance
Cabos e adaptadores (SMA)	Conforme a sua antena e filtro
Rede local (Wi-Fi ou cabo) com internet	Cabo é mais estável
Conta no Telegram	Para criar o bot e receber alertas

---

2. Como o sistema funciona

```
Antena (1090 MHz)
   |
   v
Filtro 1090 MHz
   |
   v
RTL-SDR (sinal de rádio -> dados digitais)
   |
   v
Raspberry Pi
   |-- readsb ............ decodifica e gera /run/readsb/aircraft.json
   |-- tar1090 ........... mapa web local (http://<IP_DO_PI>/tar1090)
   |-- adsbexchange-feed . envia seus dados para a rede ADS-B Exchange
   '-- radar-bot ......... lê o aircraft.json e avisa no Telegram
```

O `readsb` é o coração do sistema: ele atualiza o arquivo `aircraft.json` a cada segundo com todas as aeronaves captadas. Os outros três serviços apenas consomem esse arquivo ou a porta de dados do `readsb`. Cada serviço roda de forma independente via **systemd**, então tudo volta sozinho depois de uma queda de energia.

---

3. Montagem do hardware

1. Conecte a **antena ao filtro 1090 MHz**.
2. Conecte a saída do filtro à entrada do **RTL-SDR**.
3. Conecte o RTL-SDR a uma porta **USB do Raspberry Pi**. Se possível use um cabo de extensão curto, para afastar o dongle do calor e do ruído do Pi.
4. Posicione a antena o mais **alto possível**, com visão livre do céu, longe de outros equipamentos eletrônicos.

> **Dica:** o posicionamento da antena é o fator que mais influencia o alcance. Nenhuma configuração de software compensa uma antena mal posicionada.

---

4. Preparar o Raspberry Pi

4.1 Gravar o sistema operacional

1. No seu computador, instale o **Raspberry Pi Imager** (raspberrypi.com/software).
2. Escolha **Raspberry Pi OS Lite (64-bit ou 32-bit)**. A versão Lite não tem interface gráfica e poupa memória.
3. Clique na engrenagem (configurações avançadas) e defina:
   - Nome do computador (hostname): por exemplo `adsb`
   - **Habilitar SSH** (autenticação por senha ou chave)
   - Usuário: `<SEU_USUARIO>` e senha forte: `<SUA_SENHA_FORTE>`

- Dados do Wi-Fi, se não for usar cabo
- Fuso horário: `America/Sao_Paulo`

4. Grave no cartão microSD, coloque no Pi e ligue.

4.2 Descobrir o IP e conectar por SSH

O IP do Pi aparece na lista de dispositivos do seu roteador. Depois conecte:

```
ssh <SEU_USUARIO>@<IP_DO_PI>
```

4.3 Atualizar o sistema

```
sudo apt update && sudo apt full-upgrade -y
sudo reboot
```

> **Segurança:** use sempre uma senha forte e nunca deixe a porta SSH exposta diretamente na internet.

---

5. Instalar o decodificador (readsb)

O instalador do projeto *wiedehopf* configura o `readsb` como serviço e já cuida de detalhes como desativar drivers de TV digital que conflitam com o RTL-SDR.

```
sudo bash -c "$(wget -nv -O - https://github.com/wiedehopf/adsb-scripts/raw/master/readsb-install.sh)"
```

Se o instalador pedir para reiniciar, faça o reboot. Depois confirme:

```
sudo systemctl status readsb
```

Deve aparecer `Active: active (running)`.

5.1 Informar a localização do receptor

A posição ajuda no cálculo de distância e é obrigatória para o MLAT funcionar bem. Edite o arquivo de configuração:

```
sudo nano /etc/default/readsb
```

Procure a linha que define `RECEIVER_OPTIONS` e `DECODER_OPTIONS`, e inclua a sua posição no campo de decoder:

```
--lat <SUA_LATITUDE> --lon <SUA_LONGITUDE>
```

Use graus decimais (exemplo de formato: `-30.123456`). Salve (`Ctrl+O`, `Enter`, `Ctrl+X`) e reinicie o serviço:

```
sudo systemctl restart readsb
```

> **Privacidade:** se não quiser expor a posição exata da sua casa, use coordenadas aproximadas. Lembre que o MLAT depende de posição precisa, então aproximar demais pode prejudicá-lo.

5.2 Verificar se há dados chegando

```
cat /run/readsb/aircraft.json | head -c 500
```

Deve aparecer um JSON com a chave `aircraft`. A lista pode estar vazia se ainda não houver aeronaves no alcance.

---

6. Instalar o mapa local (tar1090)

```
sudo bash -c "$(wget -nv -O - https://github.com/wiedehopf/tar1090/raw/master/install.sh)"
```

Ao final, abra no navegador de um computador da mesma rede:

```
http://<IP_DO_PI>/tar1090
```

Você verá as aeronaves captadas em tempo real.

---

7. Alimentar o ADS-B Exchange

Este passo envia os dados da sua antena para a rede global do ADS-B Exchange. É opcional para o bot, mas contribui com a comunidade.

```
curl -L -o /tmp/axfeed.sh https://www.adsbexchange.com/feed.sh
sudo bash /tmp/axfeed.sh
```

O instalador vai pedir:

- **Latitude e longitude** da antena (graus decimais)
- **Altitude** da antena em metros
- **Nome da estação** (`<NOME_DA_SUA_ESTACAO>`), sem espaços ou caracteres especiais

Confirme que o serviço subiu:

```
sudo systemctl status adsbexchange-feed
```

Para conferir se o seu feed está sendo recebido e se o MLAT sincronizou, acesse:

```
https://www.adsbexchange.com/myip/
```

> **Normal:** logo após o boot você pode ver mensagens de “Name resolution failed” no log do feeder. É só o DNS ainda não estar pronto; ele reconecta sozinho em segundos.

---

8. Criar o bot do Telegram

8.1 Criar o bot e obter o token

1. No Telegram, procure por **@BotFather** e inicie uma conversa.
2. Envie `/newbot` e siga as perguntas (nome e nome de usuário do bot).
3. O BotFather vai responder com o **token**, algo como `<SEU_TOKEN_AQUI>`. Guarde em local seguro.

> **Atenção:** o token dá controle total sobre o bot. Se ele vazar, revogue em **@BotFather -> /mybots -> seu bot -> API Token -> Revoke current token**.

8.2 Criar o grupo e obter o Chat ID

1. Crie um grupo no Telegram e **adicione o seu bot** a ele.
2. Envie uma mensagem qualquer no grupo.
3. No Pi, consulte as atualizações do bot (substituindo o token):

```
curl "https://api.telegram.org/bot<SEU_TOKEN_AQUI>/getUpdates"
```

1. Na resposta, procure `"chat":{"id":` e anote o número. Para grupos ele costuma ser **negativo**, por exemplo `<SEU_CHAT_ID>` no formato `-1234567890`.

---

9. Instalar o bot de alertas

9.1 Criar a pasta e instalar a dependência

```
mkdir -p ~/radar_bot
cd ~/radar_bot
sudo apt install -y python3-requests
```

9.2 Colocar os arquivos na pasta

Os arquivos necessários estão no **Apêndice A** (código do bot) e **Apêndice B** (lista de aeronaves e serviço). Há duas formas de levá-los ao Pi:

- **Opção A (recomendada):** copiar do seu computador com `scp`, usando o pacote de arquivos que acompanha este guia:

```
scp radar_bot.py aeronaves.json radar-bot.service <SEU_USUARIO>@<IP_DO_PI>:~/radar_bot/
```

- **Opção B:** criar cada arquivo com `nano` e colar o conteúdo. Cuidado: colar texto grande em terminal pode bagunçar a indentação do Python. Depois de colar, sempre valide (passo 9.4).

9.3 Criar o arquivo de credenciais (.env)

```
cd ~/radar_bot
nano .env
```

Conteúdo (substitua pelos seus valores):

```
TG_TOKEN=<SEU_TOKEN_AQUI>
TG_CHAT_ID=<SEU_CHAT_ID>
```

Proteja o arquivo para que só o seu usuário consiga lê-lo:

```
chmod 600 .env
```

9.4 Validar os arquivos

```
python3 -m py_compile radar_bot.py && echo "Python OK"
python3 -m json.tool aeronaves.json > /dev/null && echo "JSON OK"
```

Se algum der erro, corrija antes de continuar.

9.5 Teste manual

```
python3 radar_bot.py
```

Você deve receber no Telegram a mensagem “Radar ADS-B Online!”. Pare o teste com `Ctrl+C`.

---

10. Deixar o bot rodando como serviço (systemd)

O serviço faz o bot iniciar sozinho no boot e reiniciar caso trave. Se o seu usuário não for `pi`, ajuste as linhas `User=` e os caminhos no arquivo antes de instalar.

```
sudo cp ~/radar_bot/radar-bot.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable radar-bot
sudo systemctl start radar-bot
sudo systemctl status radar-bot
```

Deve aparecer `active (running)`. Para acompanhar os registros ao vivo:

```
sudo journalctl -u radar-bot -f
```

---

11. Como cadastrar e editar aeronaves

Toda a lista fica em `~/radar_bot/aeronaves.json`. O bot recarrega esse arquivo sozinho a cada 5 minutos, **sem precisar reiniciar**.

```
nano ~/radar_bot/aeronaves.json
```

Cada aeronave é um bloco:

```
{"nome": "NOME DE EXIBIÇÃO", "callsign": "PREFIXO", "hex": ["CODIGO_HEX"]}
```

Regras importantes:

- `callsign` é o prefixo da matrícula **sem hífen** (exemplo: `PR-ABC` vira `PRABC`).
- `hex` é sempre uma **lista**, mesmo com um só código. Se a aeronave tem mais de um hex, coloque todos: `["AAAAAA", "BBBBBB"]`.
- Se não souber o hex, use lista vazia `[]` e a detecção será só pelo callsign.
- Entre um bloco e outro vai **vírgula**; o **último** bloco não tem vírgula.
- Use sempre aspas duplas retas (`"`).
- Evite repetir o mesmo callsign em duas entradas: só a última valerá.

Depois de editar, valide:

```
python3 -m json.tool ~/radar_bot/aeronaves.json > /dev/null && echo "JSON válido!"
```

Para aplicar na hora, sem esperar os 5 minutos:

```
sudo systemctl restart radar-bot
```

Onde achar o código hex: sites como adsbexchange.com e flightaware.com permitem buscar pela matrícula; o hex também aparece no seu mapa local quando a aeronave está no ar.

---

12. Teste final: reinício completo

Confirme que tudo volta sozinho depois de uma queda de energia:

```
sudo reboot
```

Aguarde cerca de 90 segundos, reconecte e verifique:

```
sudo systemctl status readsb adsbexchange-feed radar-bot
```

Os três devem estar `active (running)` e você deve receber “Radar ADS-B Online!” no Telegram sem digitar nada.

---

13. Comandos do dia a dia

O que fazer	Comando
Ver status de um serviço	`sudo systemctl status radar-bot`
Reiniciar o bot	`sudo systemctl restart radar-bot`
Parar / iniciar o bot	`sudo systemctl stop radar-bot` / `sudo systemctl start radar-bot`
Ver registros do bot ao vivo	`sudo journalctl -u radar-bot -f`
Editar a lista de aeronaves	`nano ~/radar_bot/aeronaves.json`
Validar a lista	`python3 -m json.tool ~/radar_bot/aeronaves.json > /dev/null && echo OK`
Ver uso de memória	`free -h`
Ver espaço em disco	`df -h`
Descobrir o IP do Pi	`hostname -I`
Reiniciar o Pi com segurança	`sudo reboot`

---

14. Problemas comuns

O bot não envia nada no Telegram Verifique `sudo journalctl -u radar-bot -n 30`. Causas comuns: token ou chat_id errados no `.env`, bot não adicionado ao grupo, ou Pi sem internet.

O serviço do bot falha ao iniciar (`status=1/FAILURE`) Quase sempre é caminho errado no `radar-bot.service` ou `.env` ausente. Confira se os arquivos estão na pasta indicada em `WorkingDirectory` e `ExecStart`.

Erro “Expecting property name enclosed in double quotes” ao validar o JSON Falta de vírgula ou de `}` em algum bloco, ou vírgula sobrando no último item. O erro costuma apontar a linha **seguinte** ao problema real; olhe também a linha anterior.

O mapa mostra “Problem fetching data from the server: JSON.parse unexpected end of data” O mapa leu o `aircraft.json` no meio de uma escrita. Se for ocasional, atualize a página. Se persistir, verifique se o `readsb` está rodando (`sudo systemctl status readsb`), se o servidor web está ativo (`sudo systemctl status lighttpd`) e a memória (`free -h`). Um reinício costuma resolver, mas se o problema voltar, o Pi pode estar no limite de recursos.

Poucas aeronaves ou alcance baixo Revise a posição da antena, o cabo, o filtro e o ganho do receptor. Em ambiente urbano, testar um ganho fixo em vez do automático às vezes melhora a recepção.

Mensagens “Name resolution failed” no feeder logo após o boot Normal; o DNS ainda não estava pronto e o feeder reconecta sozinho.

---

15. Segurança — resumo

- Nunca coloque token, senha ou chat_id dentro do código ou em prints compartilhados.
- Mantenha o `.env` com permissão `600` e fora de qualquer repositório público.
- Troque a senha padrão do usuário do Pi e use uma senha forte.
- Não exponha o SSH diretamente na internet; se precisar de acesso remoto, use uma VPN.
- Se algum segredo vazar, revogue e gere outro imediatamente.
- Mantenha o sistema atualizado: `sudo apt update && sudo apt full-upgrade` .

Apêndice A — Código do bot (`radar_bot.py`)

Salve este arquivo em `~/radar_bot/radar_bot.py`. Ele **não contém credenciais**: token e chat_id são lidos do arquivo `.env`.

```
import requests
import time
import json
import socket
import os
import logging
from pathlib import Path

# --- CONFIGURAÇÕES GERAIS ---
BASE_DIR = Path(__file__).resolve().parent
AIRCRAFT_JSON = "/run/readsb/aircraft.json"
AERONAVES_JSON = BASE_DIR / "aeronaves.json"
ENV_FILE = BASE_DIR / ".env"

INTERVALO_LOOP = 10          # segundos entre leituras do aircraft.json
INTERVALO_RECARGA_DB = 300   # segundos entre recarregar aeronaves.json (5 min)
JANELA_ANTISPAM = 1800       # 30 minutos sem repetir alerta da mesma aeronave

# --- LOGGING ---
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s",
)
log = logging.getLogger("radar_bot")


# --- CARREGAR VARIÁVEIS DE AMBIENTE (.env simples, sem depender de lib externa) ---
def carregar_env(caminho: Path) -> None:
    if not caminho.exists():
        return
    with open(caminho, "r") as f:
        for linha in f:
            linha = linha.strip()
            if not linha or linha.startswith("#") or "=" not in linha:
                continue
            chave, valor = linha.split("=", 1)
            os.environ.setdefault(chave.strip(), valor.strip())


carregar_env(ENV_FILE)

TOKEN = os.environ.get("TG_TOKEN")
CHAT_ID = os.environ.get("TG_CHAT_ID")

if not TOKEN or not CHAT_ID:
    log.error(
        "TG_TOKEN e/ou TG_CHAT_ID não configurados. "
        "Copie .env.example para .env e preencha os valores."
    )
    raise SystemExit(1)


# --- BANCO DE AERONAVES (externo, recarregável sem reiniciar o bot) ---
def carregar_banco_aeronaves(caminho: Path) -> dict:
    """
    Lê aeronaves.json (lista de {nome, callsign, hex}) e monta um dicionário
    único de busca por callsign OU hex, ambos em maiúsculas.
    O campo "hex" aceita tanto uma string única quanto uma lista de códigos
    (útil quando a mesma aeronave tem mais de um hex registrado/histórico).
    """
    banco = {}
    try:
        with open(caminho, "r", encoding="utf-8") as f:
            lista = json.load(f)
        for item in lista:
            nome = item.get("nome", "").strip()
            callsign = item.get("callsign", "").strip().upper()
            hex_valor = item.get("hex", [])
```

```
            if isinstance(hex_valor, str):
                hex_lista = [hex_valor] if hex_valor.strip() else []
            elif isinstance(hex_valor, list):
                hex_lista = hex_valor
            else:
                hex_lista = []

            if not nome:
                continue
            if callsign:
                banco[callsign] = nome
            for h in hex_lista:
                h = h.strip().upper()
                if h:
                    banco[h] = nome
        log.info(f"Banco de aeronaves carregado: {len(lista)} aeronaves cadastradas.")
    except FileNotFoundError:
        log.error(f"Arquivo {caminho} não encontrado.")
    except json.JSONDecodeError as e:
        log.error(f"Erro ao ler {caminho} (JSON inválido): {e}")
    return banco


def enviar_telegram(mensagem: str, tentativas: int = 3) -> bool:
    url = f"https://api.telegram.org/bot{TOKEN}/sendMessage"
    payload = {"chat_id": CHAT_ID, "text": mensagem, "parse_mode": "Markdown"}
    for tentativa in range(1, tentativas + 1):
        try:
            r = requests.post(url, data=payload, timeout=10)
            if r.status_code == 200:
                return True
            log.warning(f"Telegram respondeu {r.status_code}: {r.text[:200]}")
        except requests.exceptions.RequestException as e:
            log.warning(f"Tentativa {tentativa}/{tentativas} falhou: {e}")
        if tentativa < tentativas:
            time.sleep(2)
    log.error("Falha ao enviar mensagem ao Telegram após todas as tentativas.")
    return False


def get_ip() -> str:
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.connect(("8.8.8.8", 80))
        ip = s.getsockname()[0]
        s.close()
        return ip
    except OSError:
        return "127.0.0.1"


def montar_mensagem(nome_exibir: str, callsign: str, hex_id: str, ac: dict, meu_ip: str) -> str:
    alt = ac.get("alt_baro", "N/A")
    velocidade = ac.get("gs", "N/A")       # ground speed, em nós
    rumo = ac.get("track", "N/A")          # direção em graus
    squawk = ac.get("squawk", "N/A")

    return (
        f"📡 *Alvo Identificado!*\n"
        f"✈️ *Identificação:* `{nome_exibir}`\n"
        f"🆔 *Callsign:* `{callsign if callsign else 'N/A'}`\n"
        f"📟 *Hex (ICAO):* `{hex_id}`\n"
        f"📏 *Altitude:* {alt} ft\n"
        f"🚀 *Velocidade:* {velocidade} kt\n"
        f"🧭 *Rumo:* {rumo}°\n"
        f"📻 *Squawk:* `{squawk}`\n\n"
        f"[Abrir Mapa Local](http://{meu_ip}/tar1090)"
    )
```

```
def limpar_avisados(avisados: dict, agora: float) -> dict:
    return {k: v for k, v in avisados.items() if agora - v < JANELA_ANTISPAM}


def main():
    avisados = {}
    meu_ip = get_ip()
    banco_aeronaves = carregar_banco_aeronaves(AERONAVES_JSON)
    ultima_recarga_db = time.time()
    ultima_limpeza = time.time()

    log.info(f"Bot iniciado (Filtro Exclusivo) no IP: {meu_ip}")
    time.sleep(2)

    enviar_telegram(
        f"🚀 *Radar ADS-B Online!*\n📍 *IP:* `{meu_ip}`\n"
        f"Monitorando {len(banco_aeronaves)} identificadores cadastrados."
    )

    while True:
        agora = time.time()

        # Recarrega o banco de aeronaves periodicamente, sem reiniciar o bot
        if agora - ultima_recarga_db > INTERVALO_RECARGA_DB:
            banco_aeronaves = carregar_banco_aeronaves(AERONAVES_JSON)
            ultima_recarga_db = agora

        # Limpa entradas antigas do dicionário de antispam
        if agora - ultima_limpeza > 3600:
            avisados = limpar_avisados(avisados, agora)
            ultima_limpeza = agora
            log.info(f"Limpeza de 'avisados' concluída. {len(avisados)} entradas ativas.")

        try:
            if not os.path.exists(AIRCRAFT_JSON):
                log.warning(f"Aguardando {AIRCRAFT_JSON}...")
                time.sleep(INTERVALO_LOOP)
                continue

            with open(AIRCRAFT_JSON, "r") as f:
                dados = json.load(f)

            for ac in dados.get("aircraft", []):
                callsign = ac.get("flight", "").strip().upper()
                hex_id = ac.get("hex", "").upper()

                if not hex_id:
                    continue

                nome_exibir = banco_aeronaves.get(callsign) or banco_aeronaves.get(hex_id)
                if not nome_exibir:
                    continue

                if hex_id not in avisados or (agora - avisados[hex_id] > JANELA_ANTISPAM):
                    msg = montar_mensagem(nome_exibir, callsign, hex_id, ac, meu_ip)
                    if enviar_telegram(msg):
                        avisados[hex_id] = agora

        except json.JSONDecodeError:
            # O readsb pode estar no meio de uma escrita do arquivo; ignora e tenta de novo no
próximo ciclo
            log.debug("aircraft.json incompleto/corrompido nesta leitura, ignorando ciclo.")
        except Exception as e:
            log.error(f"Erro inesperado no loop: {e}")

        time.sleep(INTERVALO_LOOP)
```

```
if __name__ == "__main__":
    main()
```

Apêndice B — Arquivos auxiliares

B.1 `aeronaves.json` (exemplo com dados fictícios)

Os nomes, prefixos e códigos abaixo são **inventados**, apenas para mostrar o formato. Substitua pelas aeronaves que você quer monitorar.

```
[
  {"nome": "EXEMPLO HELICÓPTERO 01", "callsign": "PPXXX", "hex": ["AAAAAA"]},
  {"nome": "EXEMPLO RESGATE 02", "callsign": "PRYYY", "hex": ["BBBBBB", "BBBBBC"]},
  {"nome": "EXEMPLO SEM HEX", "callsign": "PSZZZ", "hex": []}
]
```

B.2 `.env.example`

```
# Copie para ".env" e preencha. NUNCA publique o arquivo ".env".
TG_TOKEN=<SEU_TOKEN_AQUI>
TG_CHAT_ID=<SEU_CHAT_ID>
```

B.3 `radar-bot.service`

Ajuste `User` e os caminhos se o seu usuário não for `pi`.

```
[Unit]
Description=Bot Radar ADS-B (Telegram)
After=network-online.target readsb.service adsbexchange-feed.service
Wants=network-online.target

[Service]
Type=simple
User=<SEU_USUARIO>
WorkingDirectory=/home/<SEU_USUARIO>/radar_bot
ExecStart=/usr/bin/python3 /home/<SEU_USUARIO>/radar_bot/radar_bot.py
Restart=always
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```